

M02 • Mon Sept. 21 / Wed Sept. 23

Code-Based Testing, Coverage

Implementation-aware testing after specification-based des

Dr. Andrew A. Allen • Fall 2026

ROADMAP

Today's focus

Move from external behavior to the structure of the implementation.



Code-based testing

Use implementation structure to guide test design and coverage analysis.



Coverage

Measure what tests exercise: statements, branches, paths, definitions, and...



OO testing

Handle classes, messages, inheritance, polymorphism, and dependencies.



CONTRAST

Specification-based vs. code-based

Different questions, different evidence.

Lens	Primary source	
Specification-based	Requirements / SRS	Expected behavior, business rules
Code-based	Source code / structure	Control flow, logic
Strong QA	Both together	Evidence that requirements are met in the implementation

Coverage answers “what ran?” —

UNIT

What is a unit?

A unit is the smallest useful piece of behavior you can test in isolation.



The unit can be a function, method, class, or small component.
A good unit test controls inputs and checks a specific expected result.
Isolation often requires stubs, fakes, mocks, fixtures, or test data.

ISOLATION

Drivers, stubs, and scaffolding

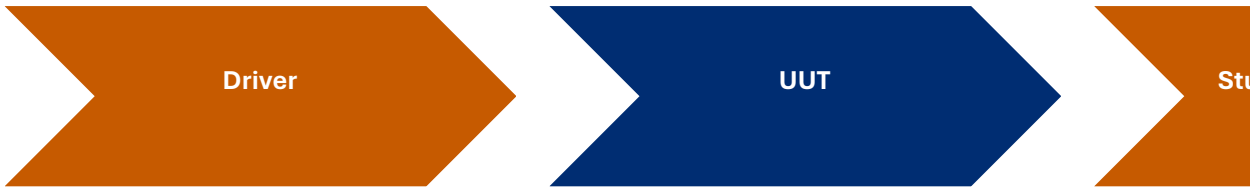
Unit testing often emulates the environment around the unit.

Test driver

Invokes the unit under test and supplies input data.

Unit under test

The function, method, class, or service evaluated.



DRIVER + DEPENDENCY

Java example: unit under test + dependency

A test driver invokes the unit; a stub/mock replaces the dependency.

```
interface ReviewerNotifier {
    void notifyReviewer(String queue, String requestId);
}

public class ApprovalRouter {
    private final ReviewerNotifier notifier;

    public ApprovalRouter(ReviewerNotifier notifier) {
        this.notifier = notifier;
    }

    public String route(Request r) {
        if (!r.isComplete()) return "RETURNED";

        String queue = !r.isTimely() ? "EXCEPTION"
            : (r.amount() > 1000 || r.isInternational())
            ? "DIRECTOR" : "COORDINATOR";

        notifier.notifyReviewer(queue, r.id());
        return queue;
    }
}
```

TEST DRIVER

JUnit 4 test method as a driver

The driver creates the setup, invokes the unit, and checks the oracle.

```
import static org.junit.Assert.*;
import org.junit.Test;

public class ApprovalRouterTest {

    @Test
    public void route_highAmount_routesToDirector() {
        // Arrange
        RecordingNotifier notifier = new RecordingNotifier();
        ApprovalRouter router = new ApprovalRouter(notifier);

        // Act
        String route = router.route(true, true, 1001,
                                    false, "PD-17");

        // Assert
        assertEquals("DIRECTOR", route);
        assertEquals("DIRECTOR", notifier.queue);
        assertEquals("PD-17", notifier.requestId);
    }
}
```

STUB / FAKE

Stub/fake dependency: predictable and isolated

A simple fake gives the unit a controlled environment and records observable interaction.

```
class RecordingNotifier implements ReviewerNotifier {
    String queue;
    String requestId;
    int callCount = 0;

    @Override
    public void notifyReviewer(String queue, String requestId) {
        this.queue = queue;
        this.requestId = requestId;
        this.callCount++;
    }
}
```

- V
- No real e needed.
 - The fake
 - The test c represents

Quick Questions

- Identify one dependency in their project that should not be used directly in a unit test.
- Examples: payment gateway, email sender, database repository, date/time provider.

MOCK / ACTIVITY

Manual mock example + in-class task

Interaction testing checks not only the return value, but also whether the right message was sent.

```
@Test
public void route_incomplete_doesNotNotify() {
    RecordingNotifier notifier = new RecordingNotifier();
    ApprovalRouter router = new ApprovalRouter(notifier);

    String route = router.route(false, true, 5000,
                                true, "PD-99");

    assertEquals("RETURNED", route);
    assertEquals(0, notifier.callCount);
}
```

- Th
- Th
- Th

- A
- A
- N
- Id

PATTERN

A small JUnit 4 unit-test pattern

Arrange → Act → Assert keeps tests readable and reviewable.

```
import static org.junit.Assert.*;
import org.junit.Test;

public class FundingRequestValidatorTest {
    @Test
    public void acceptsMinimumAmount() {
        FundingRequest request = RequestBuilder.valid()
            .amount(25).build();

        ValidationResult result = validator.validate(request);

        assertTrue(result.isValid());
        assertEquals("Submitted", result.nextStatus());
    }
}
```

CONTROL FLOW

Control flow and path testing

The code creates possible routes through a unit.

```
public String approvalPath(boolean complete,
                           boolean timely,
                           int amount,
                           boolean international) {
    if (!complete) {
        return "return";
    }
    if (!timely) {
        return "exception";
    }
    if (amount > 1000 || international) {
        return "director";
    }
    return "coordinator";
}
```

Stru

Whic

Test

Exec
teste

CONTROL FLOW

From Java code to paths

Use the implementation structure to identify decisions and branch outcomes.

```
public class ApprovalRouter {  
    public String route(boolean complete, boolean timely,  
                        int amount, boolean international) {  
        if (!complete) {  
            return "RETURN";  
        }  
        if (!timely) {  
            return "EXCEPTION";  
        }  
        if (amount > 1000 || international) {  
            return "DIRECTOR";  
        }  
        return "COORDINATOR";  
    }  
}
```

Important point

Coverage asks which routes were exercised. It does not tell us whether the requirements were

GROUP ACTIVITY

Control-flow testing with JUnit 4

Goal: cover every return path and explain what each test proves.

Task for each group

- 1. Name the four reachable return paths.
- 2. Write one JUnit 4 test for each path.
- 3. Identify which branch outcomes are still not explicit.
- 4. Link one test to a requirement or risk.

Path	Example test input	Expected
P1	complete = false	RETURN
P2	complete=T, timely=F	EXCEPTION
P3	T,T, amount=1001,F	DIRECTOR
P4	T,T, amount=800,F	COORDINATOR

```
@Test
public void incompleteRe
    ApprovalRouter router

    String result = router

    assertEquals("RETURN",
}

@Test
public void timelyDomest
    ApprovalRouter router
    assertEquals("COORDINA
        router.route(true,
    }
```

Debrief

Would these four tests catch

COVERAGE

Coverage metrics: useful but limited

Coverage helps reveal gaps, but it is not a release guarantee.

Metric	What it asks
Statement	Did each statement execute?
Branch / decision	Did each decision outcome occur?
Path	Did selected paths execute?
Condition	Did each Boolean condition vary?
Def-use	Did definitions reach uses?

COVERAGE GAPS

What coverage can and cannot prove

A test can execute code and still miss important behavior.

Example: 100% statement coverage, weak oracle

```
public boolean needsDirector(int amount, boolean international) {  
    // Requirement: director if amount > 1000 OR international.  
    return amount >= 1000 || international; // BUG: should be >  
}
```

Possible tests

Test	Input	Expected	Result
T1	amount=1500, intl=false	true	pass
T2	amount=500, intl=false	false	pass
T3 missing	amount=1000, intl=false	false	would fail

Sta
Both
the
rem

Ora
Ass
the
not

Note:
still d

HIDDEN LOGIC

Code may reveal behavior the SRS did not

Implementation-aware testing helps find undocumented, risky, or accidental behavior

```
public int normalizeAmount(int amount) {  
    if (amount < 25) throw new IllegalArgumentException("too low");  
    if (amount > 5000) return 5000; // hidden behavior  
    return amount;  
}
```

Why this matters

If the SRS says amounts above \$5,000 must be rejected, silently capping the amount is a defect even if the test passes.

What to ask

Is this behavior documented?
Does it make sense?
Could it be a defect?
Should we test for it?

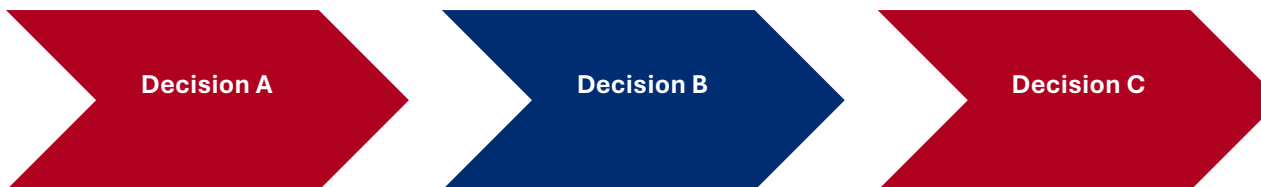
Test cases

amount = 5001
the test passes

PATHS

Path explosion: why complete path testing fails

Loops and combinations can create an infeasible number of paths.



Every branch or loop multiplies possible execution paths.
Some paths may be logically impossible or infeasible.
Choose paths based on risk, coverage criteria, and known defects.

Complete path testing is rarely realistic. De

COMPLEXITY

Cyclomatic complexity: a planning signal

More independent paths usually mean more testing attention.

Simple idea

Cyclomatic complexity estimates the number of linearly independent paths through a control-flow graph.

Signal	
Low complexity	Basic unit test
Moderate complexity	Branch coverage
High complexity	Risk review, path

DATA FLOW

Data-flow testing: definitions and uses

Faults often hide where values are created, changed, and consumed.



A definition creates or assigns a value.

A use consumes the value in a calculation, decision, output, or data transfer.

Data-flow tests ask whether important definition-use paths are exercised.

OO TESTING

Object-oriented testing adds new risks

Classes communicate through messages; inheritance and polymorphism complicate coverage.

Encapsulation

Internal state is hidden, so tests must observe behavior through public interfaces.

Composition

Classes depend on other classes; integration and mock design matter.

Inheritance

Inheritance complicates testing each class.

Unit tests should cover class behavior and selected class interactions.
Coverage across one class may not cover behavior in related subclasses.

DATA FLOW

Definitions, transformations, and use

Follow where a value is assigned, changed, tested, and returned.

```
public int finalPrice(int subtotal, boolean member) {  
    int discount = 0;           // def: discount  
    if (member && subtotal >= 100) {  
        discount = 10;         // re-def: discount  
    }  
    int total = subtotal - discount; // use: subtotal, discount  
    if (total < 0) {             // predicate use: total  
        return 0;  
    }  
    return total;               // computational use: total  
}
```

GROUP ACTIVITY

Find the data-flow and coverage gap

Goal: add focused JUnit 4 tests that exercise meaningful definitions and uses.

Given this test...

```
@Test
public void nonMemberSmallSubtotal_noDiscount() {
    PriceService svc = new PriceService();
    assertEquals(50, svc.finalPrice(50, false));
}
```

Expected a

publ

Group tasks

1. Which definition of discount did this test use?
2. Which definition is still uncovered?
3. Write one JUnit 4 test for the missing def-use path.
4. State the coverage gap in one sentence.

Debrief

The new test
threshold be
evidence tog

DEPENDENCIES

Mocks, fakes, and stubs

Dependency control is how we isolate behavior and force hard-to-reach conditions.

Use a stub/fake when...

- You need predictable data
- The real service is slow/unavailable
- You want to avoid side effects
- You need repeatable setup

Use

- You m
- The u
- You n
- You n

JUNIT 4

JUnit 4 essentials for this module

Use the framework to make expectations executable.

Test structure

@Test marks a test method
@Before prepares shared fixtures
Arrange → Act → Assert keeps the purpose clear

Core assertions

assertEquals(expected, actual)
assertTrue(condition)
assertFalse(condition)
assertNotNull(value)

```
@Test
public void approvalPath_routesDomesticLargeRequestToDirector() {
    assertEquals("director", router.approvalPath(true, true, 1001,
}
```

GROUP ACTIVITY 1

Branch coverage with ApprovalRouter

Goal: cover every branch and connect each test to an expected result.

```
public class ApprovalRouter {  
    public String approvalPath(boolean complete,  
                               boolean timely,  
                               int amount,  
                               boolean international) {  
        if (!complete) return "return";  
        if (!timely) return "exception";  
        if (amount > 1000 || international) return "director";  
        return "coordinator";  
    }  
}
```

GROUP ACTIVITY 2

OO isolation with a fake dependency

Goal: isolate one class while still checking interaction behavior.

```
interface Notifier {
    void notify(String user, String message);
}

class FakeNotifier implements Notifier {
    int calls = 0;
    public void notify(String user, String message) {
        calls++;
    }
}

@Test
public void approvalSendsOneNotification() {
    FakeNotifier fake = new FakeNotifier();
    RequestWorkflow workflow = new RequestWorkflow(fake);
    workflow.approve("requester1");
    assertEquals(1, fake.calls);
}
```

GROUP ACTIVITY 3

JUnit 4 mini-lab deliverable

Turn code-based testing into project evidence, not just a passing test file.

1 Pick a target

Choose one Java method/class with meaningful branches or dependency behavior

2 Write

Add at least one test case

3 Run coverage

Record statement/branch coverage for the target or explain why coverage is unavailable

4 Submit

Test file and coverage report

Team report-out: one test you added, one coverage gap you identified

TEST QUALITY

False pass and false fail

Tests are software too; they can be wrong.

False pass

The test passes even though the application behavior is wrong. Often caused by a weak oracle or incomplete assertion.

Fa

The
cau

Review test code with the same seriousness

EXERCISE

Exercise: write a small JUnit 4 unit-test plan

Use one Java validation, routing, or workflow method from your assigned project.

1

Select a unit

Pick a function/method with visible inputs and outputs.

2

Name the oracle

Define expected output, state change, exception, or dependency call.

4

Write 3 tests

One normal case, one boundary/invalid case, one branch/interaction case.

5

PROJECT

Project connection: what evidence should team

Unit testing artifacts should feed the project, not sit separately.

Artifact	Minimum evidence
Unit test file	Readable tests, names, assertions
Coverage report	Before/after or focused metrics
Traceability link	Requirement / risk / code area
Defect report	Steps, expected, actual, evidence
Known gap	What was not covered and why

PITFALLS

Common mistakes to catch early

These will weaken project submissions later.

Using coverage as a substitute for expected results.

Testing only the happy path.

Treating mocks as if they prove the real integration works.

Writing tests that depend on order, time, or database state without control.

Reporting “bug found” without requirement, expected result, actual result.

SUMMARY

Takeaways

Code-based testing complements specification-based testing.

1

Coverage reveals what ran

It helps find gaps but cannot prove correctness alone.

2

Unit tests need oracles

Automated tests are only useful if expected results are explicit.

Exit question: Which project component most needs testing?

SOURCES

References and alignment

Course materials and textbook concepts used in this deck.

Aligned with Lecture 5 dynamic unit testing, drivers, stubs, scaffolding, and integration transitions.
Aligned with Chapter 8 Path Testing: program graphs, paths, path explosion, and coverage.
Aligned with Chapter 9 Testing Object-Oriented Software: encapsulation, inheritance, polymorphism.
Builds on Chapters 5–7 specification-based design previously covered.
Includes Java examples and JUnit 4 group activities for implementation-aware test design.

Added Java/JUnit 4 examples for control flow, data flow, coverage gaps, and hidden logic.